

WEEK 5: Exploratory Data Analysis (EDA) & Data Wrangling Using Python (Pandas)

EDA – Exploratory Data Analysis:

Understanding the dataset before doing analysis or modelling.

The main objective of Week 5 is to **prepare raw datasets for analysis** by performing:

- Data cleaning
- Data transformation
- Filtering
- Aggregation
- Feature creation

This process is called **Data Wrangling**.

In simple words:

EDA is the process of:

- Looking at the data
- Cleaning the data
- Finding patterns
- Understanding problems in the dataset

before using it for dashboards, machine learning, or predictions.

Tools & Technologies Used

- **Google Colab** – Python execution environment
- **Python Libraries**
 - Pandas
 - NumPy
- **Dataset Type** – Multi-sheet Excel file

Dataset Description

File Name:

finanace_Dashboard.xlsx

Available Sheets:

- Monthly_closee
 - Dashboard
 - Volume_By_Name
 - Trend_Count
 - **finance_Cleaned (Raw Dataset)**
-

Selected Sheet for Week 5:

finance_Cleaned

This sheet contains raw financial data suitable for data cleaning and analysis.
Other sheets are aggregated dashboard outputs and are not suitable for wrangling.

Dataset Columns

The dataset contains the following fields:

- date – transaction date
- name – company or stock name
- open – opening price
- high – highest price
- low – lowest price
- close – closing price
- volume – traded quantity
- year – year of transaction
- month – month of transaction

DATA CLEANING USING PANDAS

Step 1: Import Required Libraries

```
import pandas as pd
```

```
import numpy as np
```

Explanation:

- Pandas is used for data manipulation.
 - NumPy helps with numerical operations.
-

Step 2: Load Excel Dataset

```
df = pd.read_excel(  
    "finanace_Dashboard.xlsx",  
    sheet_name="finance_Cleaned"  
)
```

Explanation:

- The dataset contains multiple sheets.
 - We explicitly select **finance_Cleaned** as it contains raw data.
-

Step 3: Initial Data Inspection

```
df.head()  
  
df.shape  
  
df.columns
```

Purpose:

- View sample records
 - Understand dataset size
 - Identify column names
-

Step 4: Standardize Column Names

```
df.columns = (  
    df.columns  
    .str.lower()  
    .str.strip()
```

```
.str.replace(" ", "_")  
)
```

Why this is required:

- Removes spaces
 - Avoids case-sensitive errors
 - Improves coding consistency
-

Step 5: Check Missing Values

```
df.isnull().sum()
```

Purpose:

- Identify empty or null values in each column
-

Step 6: Handle Missing Values

Numeric Columns – Median Method

```
num_cols = df.select_dtypes(include='number').columns
```

```
for col in num_cols:
```

```
    df[col].fillna(df[col].median(), inplace=True)
```

Categorical Columns – Mode Method

```
cat_cols = df.select_dtypes(include='object').columns
```

```
for col in cat_cols:
```

```
    df[col].fillna(df[col].mode()[0], inplace=True)
```

Explanation:

- Median is best for numeric data.
- Mode is best for categorical data.

Step 7: Remove Duplicate Records

```
df.duplicated().sum()
```

```
df = df.drop_duplicates()
```

Purpose:

- Prevent incorrect calculations
 - Ensure data accuracy
-

Step 8: Data Type Conversion

```
df['date'] = pd.to_datetime(df['date'])
```

Explanation:

- Converts date column into proper datetime format.
 - Enables time-based filtering.
-

DATA MANIPULATION & FILTERING

Step 9: Filtering Data

```
df_2023 = df[df['year'] == 2023]
```

Purpose:

- Extract records for a specific year.
-

Step 10: Slicing Using loc and iloc

```
df.loc[0:5]
```

```
df.iloc[0:5, 0:4]
```

Explanation:

- loc → label-based selection
 - iloc → index-based selection
-

Step 11: Creating New Calculated Columns

Price Change Calculation

```
df['price_change'] = df['close'] - df['open']
```

Profit / Loss Classification

```
df['trend'] = df['price_change'].apply(  
    lambda x: 'Profit' if x > 0 else 'Loss'  
)
```

Purpose:

- Demonstrates business logic implementation
- Converts numeric data into meaningful insights

GROUPING & MERGING

Step 12: GroupBy Operations

Average Closing Price per Company

```
df.groupby('name')['close'].mean()
```

Total Volume by Company

```
df.groupby('name')['volume'].sum()
```

Step 13: Pivot Table

```
pd.pivot_table(  
    df,  
    values='close',  
    index='name',  
    columns='trend',  
    aggfunc='mean'  
)
```

GroupBy is used to group similar data together and perform summary calculations on each group.

Purpose:

- Summarizes large datasets
- Enables multi-dimensional analysis

Step 14: Merging DataFrames

```
jan = df[df['month'] == 1]
feb = df[df['month'] == 2]
merged_df = pd.concat([jan, feb])
```

Explanation:

- Combines multiple subsets into one dataset

FINAL OUTPUT

Step 15: Export Clean Dataset

```
df.to_csv("week5_finance_cleaned.csv", index=False)
```

Result:

- Final dataset is clean
- Ready for modeling or visualization

Conclusion

In Week 5, a complete **data wrangling cycle** was performed using Python Pandas on a multi-sheet finance dataset. Raw transactional data was cleaned, transformed, filtered, and aggregated to prepare a high-quality dataset suitable for further analysis, machine learning, or dashboard development.